

Practical Insights into Data Analysis and Machine Learning

PROGRAM - 3

Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

Objective

To implement Principal Component Analysis (PCA) to reduce the dataset's dimensionality from large features to small principal components, enabling visualization of the data in a lower-dimensional space.

3. Introduction

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible.

In this implementation, we apply PCA to the classic Iris dataset, reducing its 4 features (sepal length, sepal width, petal length, and petal width) to just 2 principal components. This transformation allows us to visualize the natural structure of the data and observe the separation between the three Iris species (setosa, versicolor, and virginica), demonstrating how effective dimensionality reduction can simplify data analysis while maintaining the most important patterns and relationships in the dataset.

3.1 Principal Component Analysis (PCA)

- Principal Component Analysis (PCA) is a technique that reduces the dimensionality of large datasets by transforming many variables into a smaller set while preserving most of the original information.
- While reducing variables inevitably sacrifices some accuracy, PCA strategically trades minor precision for significant simplification. This creates datasets that are more manageable to explore and visualize, enabling machine learning algorithms to process data more efficiently by eliminating unnecessary variables.
- In essence, PCA aims to minimize the number of variables in a dataset while maximizing the retention of important information.

Principal Components

Principal components are newly constructed variables formed as linear combinations of the original variables. These combinations are designed with two key properties:

1. The new variables (principal components) are uncorrelated with each other
2. Information from the original dataset is distributed optimally, with the first component capturing the maximum possible variance, the second component capturing the maximum remaining variance, and so on

In practice, this means that when analyzing 10-dimensional data, PCA will generate 10 principal components, but the information is redistributed so that earlier components contain more information than later ones. This approach allows analysts to focus on the first few components that contain most of the dataset's information, effectively achieving dimensionality reduction while minimizing information loss.

3.2 Calculating Principal Components

The PCA computation process follows these steps, showing how principal components are calculated and relate to the original data:

1. **Standardize the Variables:** Standardization is essential prior to performing PCA, as the technique is sensitive to the relative scaling of the original variables. We transform the dataset by centering all variables to a *mean of zero* and *standard deviation of 1*, preventing features with larger numerical ranges from disproportionately influencing the principal components.

Formula for standardization (Z-score normalization):

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

where:

X is the original value

μ is the mean of the variable

σ is the standard deviation

2. **Compute the Covariance Matrix:** The covariance matrix measures how variables are correlated with each other. If two variables have a high covariance, it means they are highly correlated.

Covariance between two variables X_i and X_j

$$\text{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{k=1}^n (X_{ik} - \bar{X}_i)(X_{jk} - \bar{X}_j)$$

The covariance matrix for a dataset with d features is:

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \dots & \text{Cov}(X_1, X_d) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \dots & \text{Cov}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_d, X_1) & \text{Cov}(X_d, X_2) & \dots & \text{Var}(X_d) \end{bmatrix}$$

3. Compute the Eigenvalues and Eigenvectors

- Eigenvalues and eigenvectors help us determine the principal components.
- Eigenvectors represent the directions of maximum variance (principal components).
- Eigenvalues indicate the amount of variance captured by each principal component.

We solve the equation:

$$\Sigma v = \lambda v$$

Σ is the covariance matrix

v is an eigenvector

λ is the corresponding eigenvalue

The eigenvectors form the principal components, and the corresponding eigenvalues show the importance of each component.

4. Sort Eigenvalues and Select Principal Components

- Arrange the eigenvalues in descending order.
- The eigenvector corresponding to the largest eigenvalue is the first principal component, the next largest is the second principal component, and so on.
- If we want to reduce dimensions, we select the top k principal components that capture the most variance.

5. Transform the Data to the New Subspace

- To obtain the transformed dataset, project the original data onto the selected principal components:

$$X_{\text{new}} = X_{\text{scaled}} \cdot V_k$$

where,

V_k is the matrix of the top k eigenvectors.

6. Choose the Number of Principal Components

- To decide how many principal components to retain, we use the explained variance ratio:

$$\text{Explained Variance Ratio} = \frac{\lambda_i}{\sum \lambda}$$

We often use a scree plot (plot of eigenvalues) or keep components that capture a certain percentage (e.g., 95%) of the variance.

3.3 Applications of PCA

- Dimensionality Reduction: Reducing the number of features in high-dimensional data.
- Data Visualization: Representing high-dimensional data in 2D or 3D plots.
- Noise Filtering: Removing less important components to improve model performance.
- Feature Extraction: Selecting the most important features in machine learning.

3.4 Program

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

```
# Load the Iris dataset
iris = load_iris()
X = iris.data # Features (4 dimensions)
y = iris.target # Labels
```

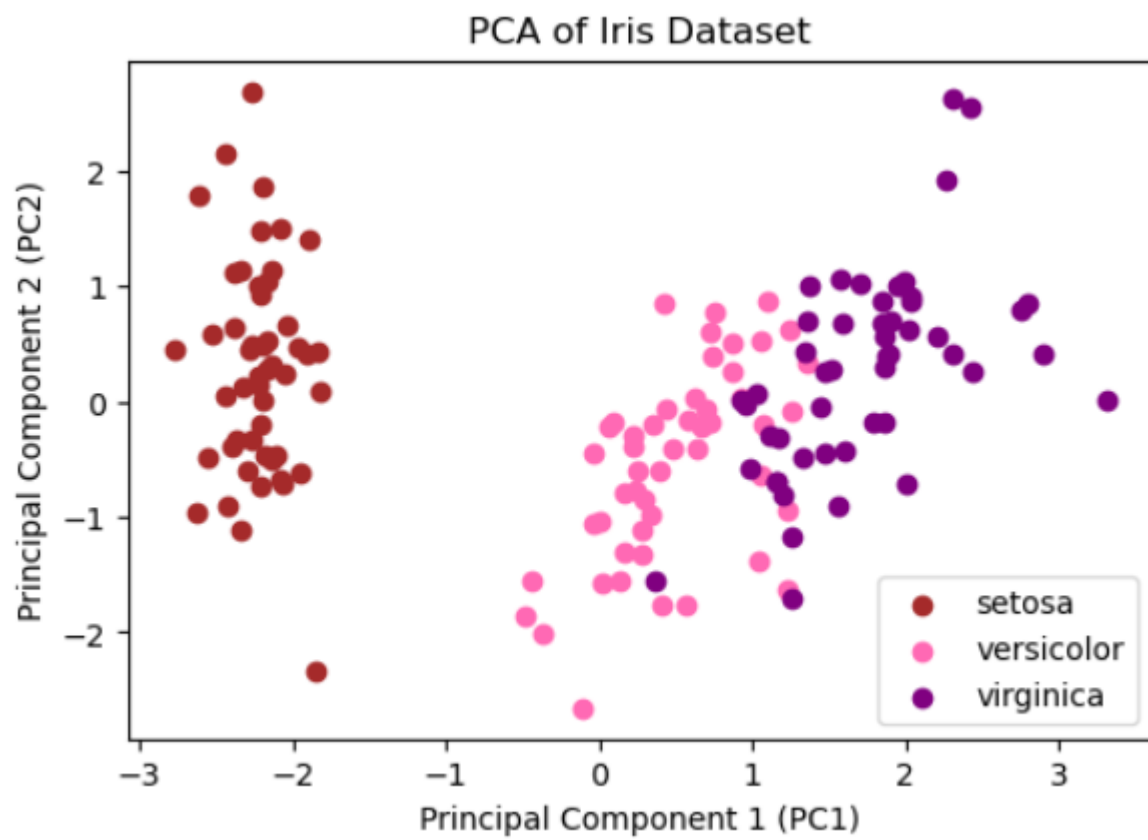
```
# Standardize the data (mean=0, variance=1)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_std = scaler.fit_transform(X)
```

```
# Apply PCA to reduce the dimensionality to 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_std)
```

```
# Create a DataFrame for the reduced data
df_pca = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])
df_pca['Species'] = y
```

```
# Visualize the reduced data
plt.figure(figsize=(6, 4))
species = ['Setosa', 'Versicolor', 'Virginica']
colors = ['brown', 'hotpink', 'purple']
for species, color in zip(np.unique(y), colors):
    plt.scatter(df_pca.loc[df_pca['Species'] == species, 'PC1'],
                df_pca.loc[df_pca['Species'] == species, 'PC2'],
                c=color, label=iris.target_names[species])

plt.xlabel('Principal Component 1 (PC1)')
plt.ylabel('Principal Component 2 (PC2)')
plt.title('PCA of Iris Dataset')
plt.legend()
plt.show()
```



Viva Questions

- What is Principal Component Analysis (PCA)?
- Why do we use PCA in machine learning?
- What are the main assumptions of PCA?
- How does PCA reduce dimensionality while retaining most of the information?
- What are eigenvalues and eigenvectors, and how are they related to PCA?
- What is the role of the covariance matrix in PCA?
- Why do we standardize the data before applying PCA?
- How do you decide how many principal components to retain?
- What is the explained variance ratio?
- What are some real-world applications of PCA?
- What happens if we do not standardize the dataset before applying PCA?